

RICORDIAMO:

LA **SYMBOL TABLE** È UNA STRUTTURA DATI USATA DALL'ASSEMBLER PER TENERE TRACCIA DEI **SIMBOLI** DEFINITI NEL PROGRAMMA.

UN SIMBOLO È:

- UNA **LABEL** DI UNA **FUNZIONE** O DI UN **BLOCCO DI CODICE**
- IL **NOME** DI UNA **VARIABILE GLOBALE**
- UNA **COSTANTE** DEFINITA CON UNA **DIRETTIVA** (**.SET**)

PER FAR SÌ CHE L'ASSEMBLER POPOLI CORRETTAMENTE LA **SYMBOL TABLE**, UTILIZZA IL **LOCATION COUNTER**, CHE TIENE TRACCIA DELL'INDIRIZZO DEL **PROSSIMO BYTE** CHE VERRÀ SCRITTO NEL PROGRAMMA.

OGNI VOLTA CHE VIENE LETTO UN'ISTRUZIONE O DATO, LC VIENE AGGIORNATO ALLA **POSIZIONE SUCCESSIVA**.

È UNA SORTA DI **CONTATORE** CHE AVANZA CON OGNI BYTE DI CODICE E OGNI VOLTA CHE VIENE DEFINITO UN **SIMBOLO** O **LABEL** VIENE RAPPATO SULL'INDIRIZZO CORRENTE DI LC NELLA **SYMBOL TABLE**.

L' **LC** PARTE DA UN **VALORE INIZIALE**, CHE È DOVE VENGONO POSIZIONATE LE **ISTRUZIONI**, SI PARTE DALLA DIRETTIVA **.TEXT** E OGNI VOLTA CHE L'ASSEMBLER LEGGE UN'ISTRUZIONE AGGIORNA LC (**DI +4** SE L'ISTRUZIONE È DA 4 BYTE).

QUANDO INCONTRA UN' **ETICHETTA**, VIENE ASSOCIATO AUTOMATICAMENTE QUELLA ETICHETTA ALL'INDIRIZZO ATTUALMENTE PUNATO DA LC, E VIENE RAPPATO NELLA **SYMBOL TABLE**.

PER I DATI È SIMILE, QUANDO L'ASSEMBLER PROCESSA LE **VARIABILI GLOBALE**, L' **LC** PARTE DA UN **VALORE INIZIALE DIVERSO** DA QUELLO DELLE ISTRUZIONI.

QUANDO L'ASSEMBLER INCONTRA UNA **DICHIARAZIONE DI DATI**, L' **LC** VIENE AGGIORNATO PER ALLOCARE **SPAZIO** NECESSARIO AI DATI STESSI.

SE SI DEFINISCE UNA **WORD** VIENE INCREMENTATO DI **+4**, CON UN **BYTE +1**, ECC.

.SKIP → È UNA DIRETTIVA USATA PER ALLOCARE SPAZIO PER VARIABILI O STRUTTURA SENZA INIZIALIZZARLO.

↳ LO FA AGGIORNANDO IL **LOCATION COUNTER**

.DATA

ARRAY:

ARRAY: **FINE:**

.BYTE 1, 2, 3 ⇒ **UNIT-8 ARRAY** [5] = { 1, 2, 3, 0, 0, 0, 0, 0, 9 }

.SKIP 5

FINE:

.BYTE 9

SYMBOL TABLE

00000000	ARRAY
00000008	FINE

.ALIGN → SERVE PER ALLINEARE I DATI O ISTRUZIONI A UN INDIRIZZO DI MEMORIA **MULTIPLO DI UNA POTENZA DI 2**. GARANTISCE EFFICIENZA E CHE I DATI SIANO DISPOSTI IN MODO CORRETTO

.DATA

VALORE 1:

.BYTE 10

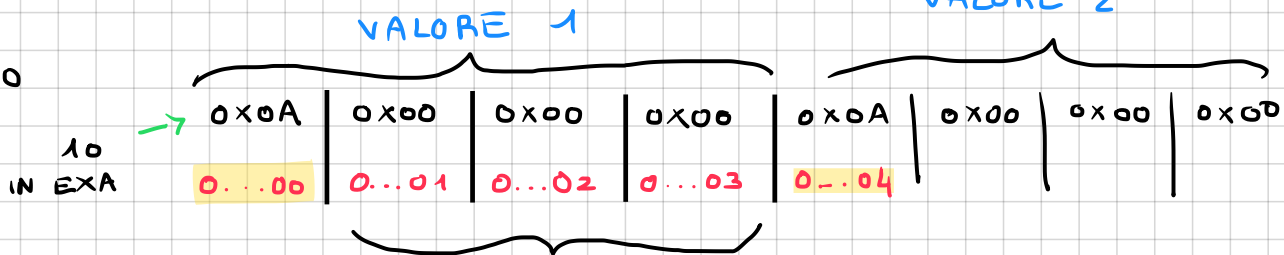
.ALIGN 2 # ALLINEA A 2^2 BYTE

VALORE 2:

SYMBOL TABLE

00000000	VALORE 1
00000004	VALORE 2

.WORD 10



VENGONO AGGIUNTI TOT PADDING
FINCHÉ NON È **MULTIPLO** DI 2^2

ABBIAMO VISTO CHE IL **LOCATION COUNTER** PARTE DA 2 **VALORI INIZIALI**

DIFFERENTI, OGNI VOLTA CHE APPARE LA DIRETTIVA **.TEXT** AVANZANO GLI **INDIRIZZI DELLE ETICHETTE** AD OGNI ISTRUZIONE, INVECE OGNI VOLTA CHE INCONTRA LA DIRETTIVA **.DATA** AVANZANO GLI **INDIRIZZI DEI SIMBOLI**, "CONGELANDO" GLI **INDIRIZZI DELLE ETICHETTE**.

NELLA **SYMBOL TABLE** NON VIENE SCRITTO L'**INDIRIZZO FISICO IN RAM**, MA SOLO L'**OFFSET**, SARÀ POI IL **LINKER** AD UNIRE TUTTE LE **SEZIONI** DI TUTTI I FILE E ASSEGNARE UN **INDIRIZZO FISICO NELLA RAM**.

.DATA

X: .BYTE 10
Y: .BYTE 11
Z: .WORD 0

METTIAMO CHE I DATI PARTONO DA 0X10000000
E LE ISTRUZIONI DA 0X00000000

X = 0X10000001
Y = 0X10000002
Z = 0X10000003

.TEXT

SERVIREBBE .ALIGN 2

LA A0, X

LB A0, 0(A0)

DIVERSI = 9 ISTRUZ. x 4 BYTE = 36₁₀ = 24₁₆ → 0X00000024

LA A1, Y

UGUALI = 13 ISTR x 4 = 52₁₀ = 34₁₆ → 0X00000034

LB A1, 0(A1)

LA A2, Z # PRENDI L'INDIRIZZO DI Z

BEQ A0, A1, UGUALI

DIVERSI:

LI A0, 1 # A0 = 1

SW A0, 0(A2) # Z = A0 = 1

LI A7, 10 # EXIT

ECALL

UGUALI: ZERO

SW x0, 0(A2) # Z = x0 = 0

LI A7, 10

ECALL

LA → 2 ISTRUZIONI (LUI + ADDI)
↓

MA DIPENDE

2) .TEXT
.GLOBL -START

METTIAMO CHE LE ISTRUZIONI PARTONO DA 0X00000000

-START:

LI x5, 10

CALL FUNC

NOP

→ TRADOTTA IN ADDI x0, x0, 0

LOOP:

ADDI x5, x5, -1

BNE x5, x0, LOOP

RET

FUNC:

MV x6, x5

RET

-START = 0X00000000 = 0₁₀
LOOP = 0X0000000C = 12₁₀
FUNC = 0X00000018 = 24₁₀

IL GCC MAGARI LO AVREBBE
TRADOTTO COME:

-START = 0X00000000 = 0₁₀
LOOP = 0X00000010 = 16₁₀
FUNC = 0X0000001C = 28₁₀

IL GCC VISTO CHE NON CONOSCE IN CHE POSIZIONE DELLA RAM IL PROGRAMMA VERRÀ MESSO, QUINDI USA $ADDI / JALR$ PER DIRE DI "ANDARE ALL'ISTRUZIONE CHE SI TROVA A X BYTE DI DISTANZA DA DOVE CI TROVIAMO ORA".

NEL SIMULATORE LA MEMORIA È FISSA QUINDI PREVEDIBILE, IL LINKER STARTA LA COMPLESSITÀ DI $ADDI$ E USA SALT DIRETTI E ASSOLUTI.

- $CALL$ → TRADOTTA IN 1 ISTRUZIONE, OVERO JAL (NEL GCC VERO SE IL CODICE È TROPPO LONTANO DIVENTA $ADDI + JALR$).
- LA → LUI (20 BIT + SIGNIFICATIVI) + $ADDI$ (12 BIT - SIGNIFICATIVI).
- LI → SE IL NUMERO STA NEI 12 BIT CON SEGNO $[-2048, +2047]$ ALLORA VIENE TRADOTTO IN 1 ISTRUZIONE, OVERO $ADDI RD, XO, IMM$, SE IL NUMERO SFORA IL RANGE, VERRÀ TRADOTTO IN 2 ISTRUZIONI, OVERO $LUI + ADDI$.

3) METTIAMO CHE I DATI PARTONO DA $0x10000000$
E LE ISTRUZIONI DA $0x00000000$

$X: .WORD 123$ $X = 0x10000000$

$.TEXT$ $-START = 0x00000000$

$.GLOBAL -START$

$-START$

$LI T1, 500$ # NUMERO DENTRO A $[-2048, +2047]$ → 1 ISTRUZIONE

$LI T2, 5000$ # NUMERO FUORI RANGE → 2 ISTRUZIONI

$ADD AO, T1, T2$

$LA TO, X$ $FUNC_OK = 0x00000024$

$LW TO, 0(TO)$

$ADD AO, AO, TO$

$CALL FUNC_OK$

$FUNC_OK:$

$ADDI AO, AO, 1$

RET